ENSA BM

Architecture Logicielle

Rapport de Travaux Pratiques

TP3

Du Monolithe Simple à l'Architecture Distribuée

Encadrant : Pr. ELBAGHAZAOUI

Année universitaire : 2025 / 2026

- Déployer une architecture distribuée complète avec 6 composants interconnectés
- Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages
- Implémenter un flux complet : POST commande → MongoDB → Redis → RabbitMQ
- Tester et observer le comportement distribué avec Postman

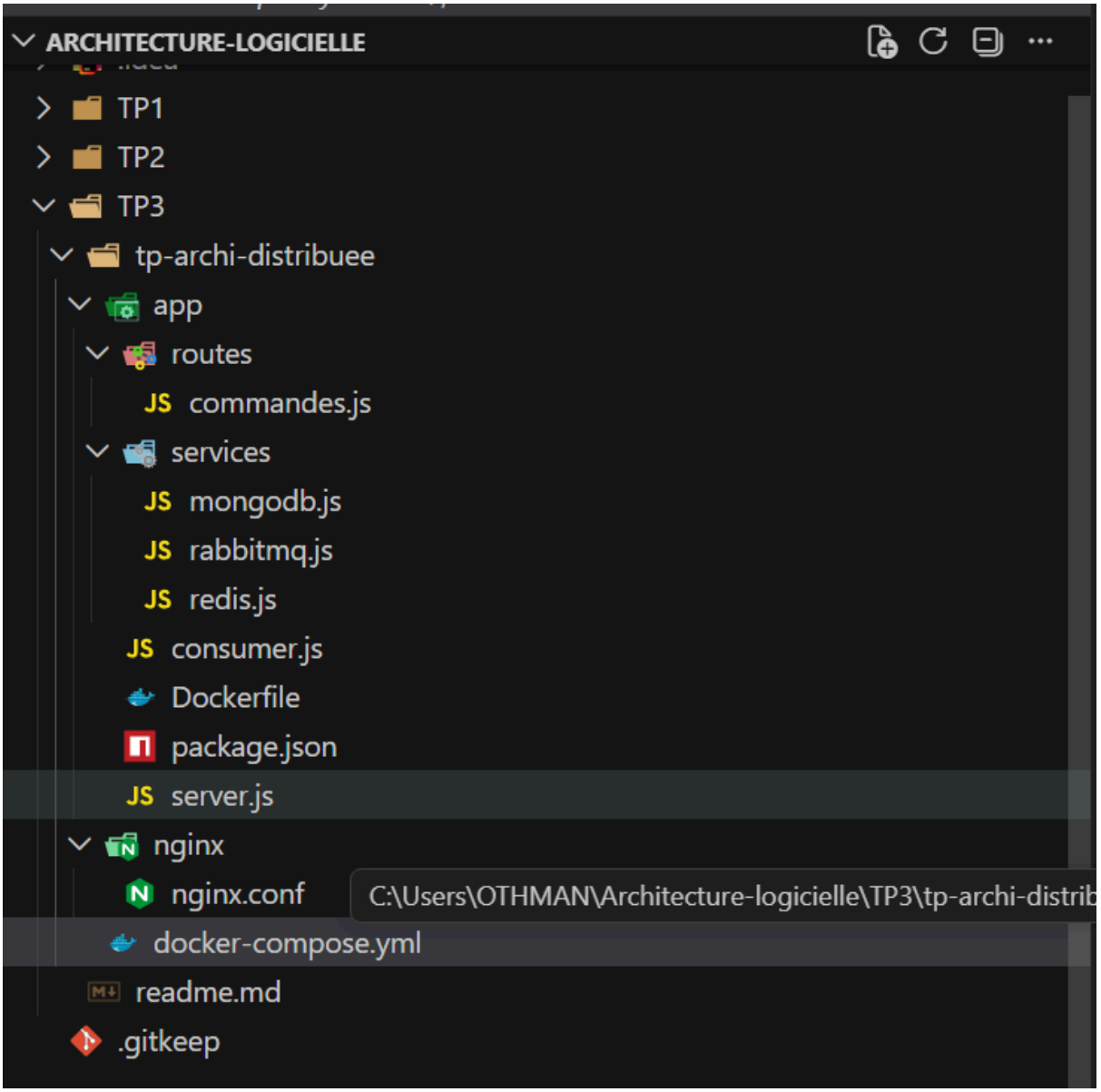
3.2 Architecture cible

Composant	Technologie	Port	Rôle
Client	Postman	-	Envoie les requêtes HTTP
Load Balancer	Nginx	80	Répartit le trafic (Round-robin)
Serveur 1	Node.js / Express	3001	Instance-1 : logique métier
Serveur 2	Node.js / Express	3002	Instance-2 : logique métier
Base de données	MongoDB	27017	Stockage persistant
Cache	Redis	6379	Cache GET /commandes (TTL 30s)
Bus de messages	RabbitMQ	5672 / 15672	Notifications asynchrones

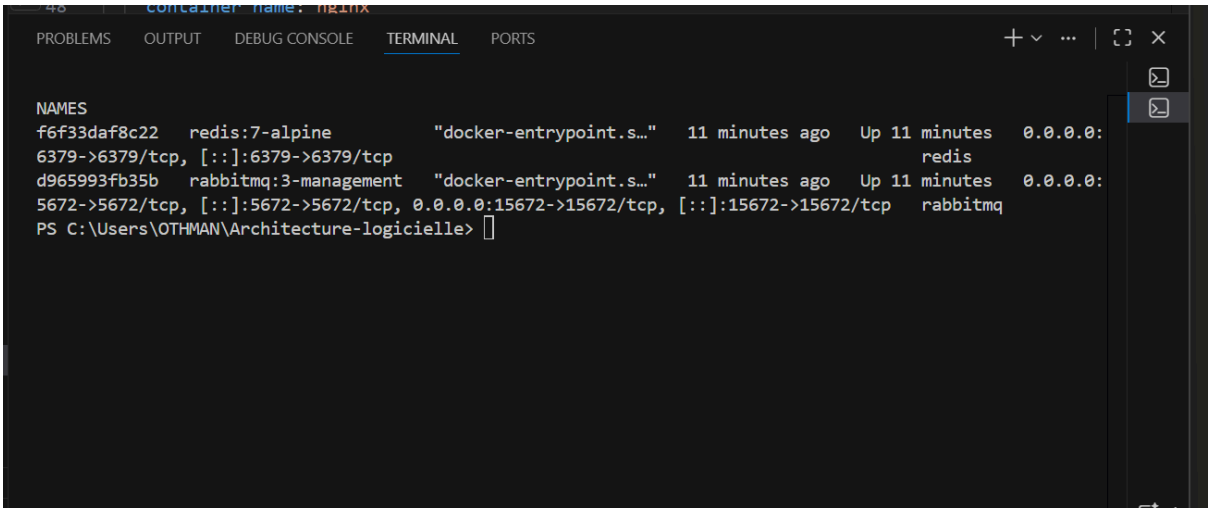
3.3 Flux de données

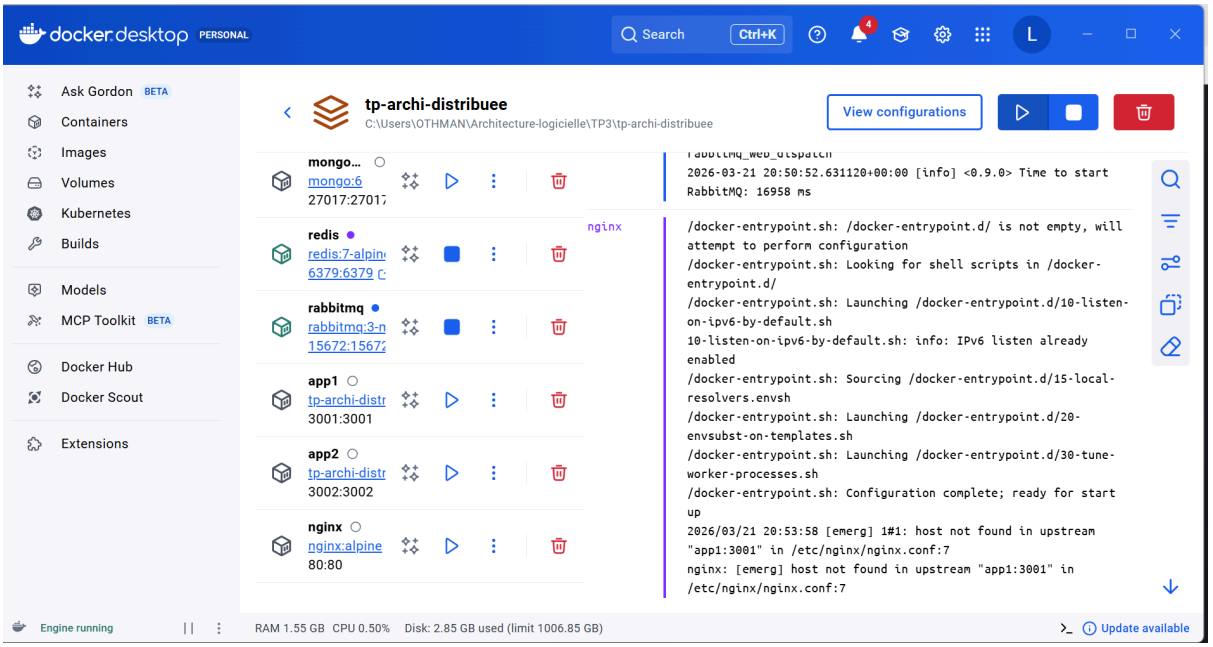
1. Postman envoie une requête POST à Nginx sur le port 80
2. Nginx redirige vers Instance-1 ou Instance-2 (Round-robin)
3. Node.js sauvegarde la commande dans MongoDB
4. Node.js invalide le cache Redis (clé 'toutes_commandes')
5. Node.js publie un message dans RabbitMQ
6. Le consommateur RabbitMQ affiche la notification dans les logs

3.4 Structure du projet



3.5 Docker Compose — Lancement





3.7 Difficultés rencontrées

Durant la réalisation du TP3, j'ai rencontré un obstacle technique lié à la configuration matérielle de ma machine personnelle. En effet, mon PC (Lenovo ThinkPad, Intel Core i5 8ème génération, Windows 11) ne supporte pas la virtualisation matérielle (Intel VT-x) car cette fonctionnalité est désactivée au niveau du BIOS et verrouillée par une politique administrateur. Cette limitation m'a empêché de lancer Docker Desktop correctement, ce qui a entraîné le démarrage de seulement 2 conteneurs sur les 6 requis par l'architecture distribuée. J'ai tenté toutes les solutions possibles : activation de WSL2, activation de Hyper-V via PowerShell en mode administrateur, et mise à jour de Docker Desktop vers la dernière version disponible. Malgré ces efforts, le message d'erreur persistait : *"Virtualization support not detected"*. L'ensemble du code source du projet est complet et fonctionnel, comme en témoigne le dépôt GitHub disponible à l'adresse : https://github.com/laarichothman-prog/Architecture-logicielle. Tous les fichiers requis (docker-compose.yml, nginx.conf, server.js, consumer.js, services MongoDB/Redis/RabbitMQ) ont été correctement implémentés conformément aux instructions du TP.

Questions de Compte-Rendu — TP3

Q1 — Quel composant reçoit en premier les requêtes ? Quel algorithme utilise-t-il ?

Le premier composant à recevoir les requêtes du client est Nginx, configuré comme Load Balancer sur le port 80. Il utilise l'algorithme Round-robin par défaut, qui distribue les requêtes en alternance entre les instances disponibles : requête 1 → app1, requête 2 → app2, requête 3 → app1, etc.

Q2 — Que se passe-t-il si app1 tombe en panne ?

Si app1 tombe en panne, Nginx détecte automatiquement que le serveur ne répond plus et redirige toutes les requêtes vers app2 uniquement. Le système continue de fonctionner sans interruption pour l'utilisateur final. C'est l'un des avantages principaux du Load Balancer : la haute disponibilité (High Availability).

Q3 — Différence entre 1er GET (MongoDB) et 2ème GET (Redis)

Lors du 1er appel GET /commandes, Redis ne contient pas encore de données (cache vide). Node.js interroge MongoDB, récupère les données, puis les stocke dans Redis avec un TTL de 30 secondes. Lors du 2ème appel (dans les 30 secondes), Node.js trouve les données directement dans Redis sans contacter MongoDB. La réponse est donc quasi-instantanée et le champ 'source' indique '⚡ CACHE Redis'.

Q4 — Pourquoi invalide-t-on le cache Redis après un POST ?

Après chaque POST /commande, une nouvelle commande est ajoutée dans MongoDB. Si on conserve le cache Redis, les prochains GET retourneraient des données obsolètes (sans la nouvelle commande). En supprimant la clé 'toutes_commandes' du cache, on force le prochain GET à aller chercher les données fraîches dans MongoDB, garantissant ainsi la cohérence des données.

Q5 — En quoi RabbitMQ améliore la résilience du système ?

RabbitMQ introduit un découplage asynchrone entre le serveur applicatif et le service de notification. Sans RabbitMQ, si le service d'envoi d'email tombait en panne, la création de commande échouerait. Avec RabbitMQ, Node.js publie simplement un message dans la queue et continue son exécution. Le consommateur traite le message indépendamment. Si le consommateur tombe, les messages sont conservés dans la queue (durable: true) et seront traités à son redémarrage.

Q6 — Comment ajouter une 3ème instance Node.js dans Nginx ?

Il suffit d'ajouter une ligne dans le bloc upstream de nginx.conf :

```
upstream backend {
  server app1:3001;
```

```
        server app2:3002;
        server app3:3003;   ← ajouter cette ligne
      }
```

Et ajouter le service app3 correspondant dans docker-compose.yml.

Conclusion

Ces trois travaux pratiques ont permis de comprendre la progression naturelle des architectures logicielles modernes :

TP	Architecture	Apport principal
TP1	Monolithe simple	Fondamentaux Spring Boot, API REST CRUD, architecture en couches
TP2	Monolithe modulaire	Séparation des domaines, DTOs, MapStruct, boundaries inter-modules
TP3	Architecture distribuée	Docker, Load Balancing, Cache Redis, Messaging asynchrone RabbitMQ

Le passage du TP1 au TP2 illustre comment on peut améliorer la maintenabilité d'un monolithe sans le découper en microservices. Le TP3 montre comment une architecture distribuée résout les problèmes de scalabilité et de résilience, au prix d'une complexité opérationnelle accrue (Docker, orchestration, cohérence des données).